

Layer 7 WAF Implementation

 Courses	 <u>NETWORK SECURITY INFRASTRUCTURE AND DESIGN</u>
☒ Done	☐

Objective: To design and deploy a custom Web Application Firewall (WAF) to protect a vulnerable e-commerce application from Reflected Cross-Site Scripting (XSS) and SQL Injection (SQLi).

1. The Architecture

The system was built using a containerized approach to simulate a real-world enterprise environment:

- **Web Server:** A custom PHP "Tech Store" (Apache) running on Docker.
- **Security Layer:** Nginx Proxy Manager (NPM) acting as a Reverse Proxy and WAF.
- **Perimeter:** Cloudflare Tunnel providing a secure, encrypted bridge to the public internet.

Phase 1: Building the Target (The Vulnerable Web App)

The first step is creating a realistic environment to protect. We used a **Server-Side Rendered (SSR)** PHP site because modern Single Page Apps (SPAs) often "hide" attacks from WAFs using client-side routing.

The Setup

1. **Code Creation:** Create an `index.php` file with an intentional Reflected XSS vulnerability in the search parameter.

```
File Edit Selection View Go Run Terminal Help
Restricted Mode is intended for safe code browsing. Trust this window to enable all features. Manage Learn More

index.php X Release Notes: 1.115.0
C:\> Users\Faez\Documents\Computer Network\Lab\Test_WAF\MyVulnerableSite> index.php
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <title>ByteBandit Tech Store | Official Gear</title>
6   <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
7   <style>
8     .hero-section { background: #f8f9fa; padding: 60px 0; }
9     .product-card { transition: transform 0.2s; }
10    .product-card:hover { transform: scale(1.05); }
11  </style>
12 </head>
13 <body>
14
15 <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
16   <div class="container">
17     <a class="navbar-brand" href="#">ByteBandit <span class="text-primary">Tech</span></a>
18     <form class="d-flex" method="GET" action="">
19       <input class="form-control me-2" type="search" name="query" placeholder="Search gadgets...">
20       <button class="btn btn-outline-primary" type="submit">Search</button>
21     </form>
22   </div>
23 </nav>
24
25 <header class="hero-section text-center">
26   <div class="container">
27     <h1 class="display-4 fw-bold">Upgrade Your Hacking Rig</h1>
28     <p class="lead text-muted">Premium hardware for the modern security researcher.</p>
29   </div>
30 </header>
31
32 <main class="container my-5">
33   <?php
34     if (isset($_GET['query'])) {
35       $search = $_GET['query'];
36       echo '<div class="alert alert-info" role="alert">';
37       // VULNERABILITY: Still echoing the raw search query (Reflected XSS)
38       echo "Showing results for: <strong>". $search . "</strong>";
39       echo '</div>';
40     }
41   <?>
42
43   <div class="row g-4">
44     <div class="col-md-4">
45       <div class="card h-100 product-card shadow-sm">
46         <div class="card-body">
47           <h5 class="card-title">Rubber Ducky V2</h5>
```

2. **Containerization:** Use Docker to deploy a PHP-Apache server.

Command used:

```
docker run -d --name my-lab -p 8080:80 -v ${PWD}:/var/www/html php:apache
```

```
PS C:\Users\Faez\Documents\Computer Network\Lab\Test_WAF\MyVulnerableSite> docker run -d --name my-lab -p 8080:80 -v ${PWD}:/var/www/html php:apache
9541e048809da0362393776f12859d2c44df46519208a78c018f07b7ad5a3962
```

Phase 2: The Bodyguard (Nginx Proxy Manager)

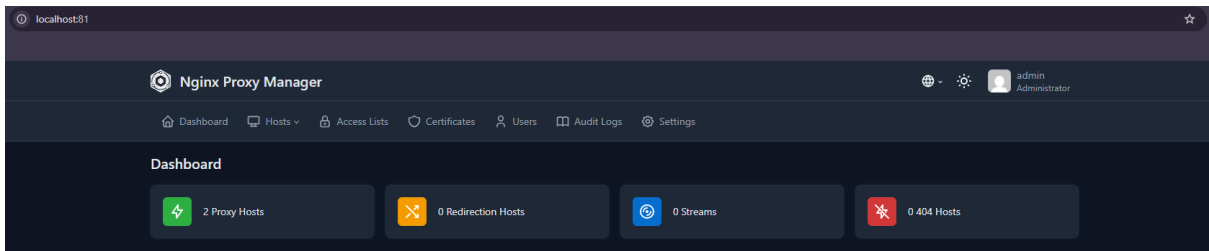
Next, we deploy the Reverse Proxy. This acts as the "Middleman" that will inspect all incoming traffic before it hits our PHP site.

The Setup

1. Deploy NPM using `docker-compose` .

```
PS C:\Users\Faezl\Documents\Computer Network\Lab\Test_WAF> docker-compose up -d
time="2026-04-11T00:48:46+08:00" level=warning msg="C:\\Users\\Faezl\\Documents\\Computer Network\\Lab\\Test_WAF\\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] up 37/37
✓Image jc21/nginx-proxy-manager:latest Pulled 74.4s
✓Network test_waf_waf-net Created 0.1s
✓Container test_waf-app-1 Started 2.0s
```

1. Access the Admin UI on port `81` .



Phase 3: The Bridge (Cloudflare Tunnel)

To make the site public and test it over the "Real Internet," we use a Cloudflare Tunnel. This creates a secure tunnel from your local machine to Cloudflare's global network.

The Setup

Command used:

```
cloudflared tunnel --url http://127.0.0.1:80
```

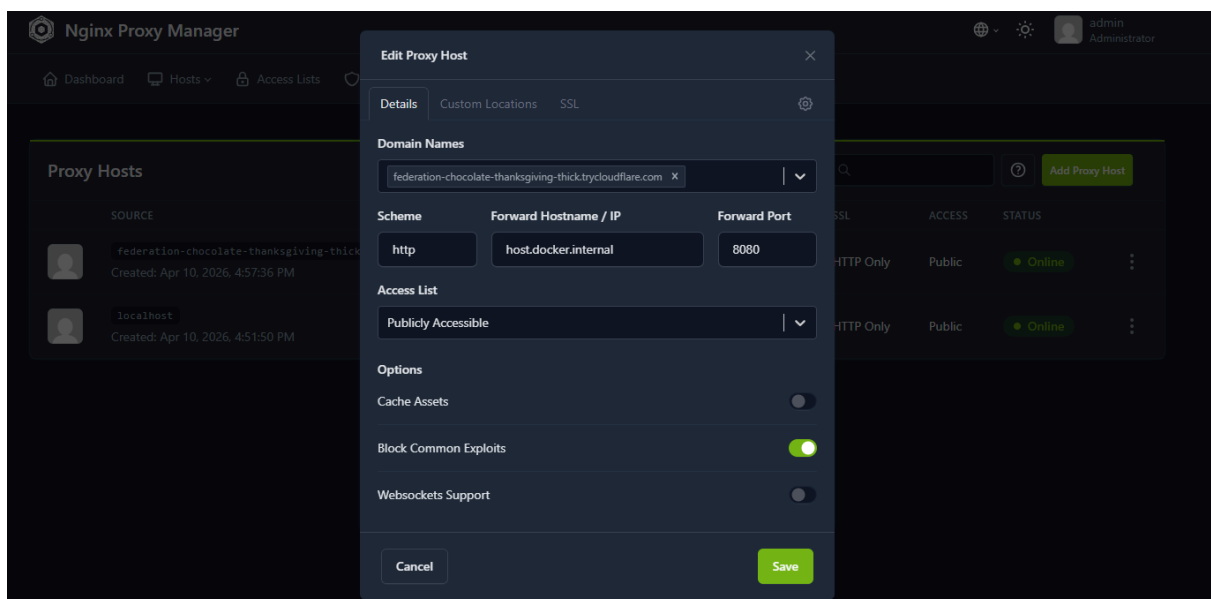
```
PS C:\Users\Faezl\Documents\Computer Network\Lab\Test_WAF\MyVulnerableSite> cloudflared tunnel --url http://127.0.0.1:80
2026-04-10T17:34:49Z INF Thank you for trying Cloudflare Tunnel. Doing so, without a Cloudflare account, is a quick way to experiment and try it out. However, be aware that these account-less Tunnels have no uptime guarantee, are subject to the Cloudflare Online Services Terms of Use (https://www.cloudflare.com/website-terms/), and Cloudflare reserves the right to investigate your use of Tunnels for violations of such terms. If you intend to use Tunnels in production you should use a pre-created named tunnel by following: https://developers.cloudflare.com/cloudflare-one/connections/connect-apps
2026-04-10T17:34:49Z INF Requesting new quick Tunnel on trycloudflare.com...
2026-04-10T17:34:54Z INF | Your quick Tunnel has been created! Visit it at (it may take some time to be reachable): |
2026-04-10T17:34:54Z INF | https://federation-chocolate-thanksgiving-thick.trycloudflare.com |
2026-04-10T17:34:54Z INF +-----+
2026-04-10T17:34:54Z INF Cannot determine default configuration path. No file [config.yml config.yaml] in [~/cloudflared ~/cloudflare-warp ~/cloudflare-warp]
2026-04-10T17:34:54Z INF Version 2025.8.1 (Checksum b5d598b00cc3a28cab5812d9f762819334614bae452db4e7f23ee7b081556)
2026-04-10T17:34:54Z INF GOOS: windows, GOVersion: go1.24.2, GoArch: amd64
2026-04-10T17:34:54Z INF Settings: map[ha-connections:1 protocol:quic url:http://127.0.0.1:80]
2026-04-10T17:34:54Z INF cloudflared will not automatically update on Windows systems.
2026-04-10T17:34:54Z INF Generated Connector ID: d8ca9909-8f9e-442b-aff1-44a8f37326f3
2026-04-10T17:34:54Z INF Initial protocol quic
2026-04-10T17:34:54Z INF ICMP proxy will use 192.168.0.11 as source for IPv4
2026-04-10T17:34:55Z INF ICMP proxy will use 2001:e68:449d:e28e:5565:48c:44b6:c516 in zone Wi-Fi 2 as source for IPv6
2026-04-10T17:34:55Z ERR Cannot determine default origin certificate path. No file cert.pem in [~/cloudflared ~/cloudflare-warp ~/cloudflare-warp]. You need to specify the origin certificate path by specifying the origincert option in the configuration file, or set TUNNEL_ORIGIN_CERT environment variable originCertPath=
2026-04-10T17:34:55Z INF cloudflared does not support loading the system root certificate pool on Windows. Please use --origin-ca-pool <PATH> to specify the path to the certificate pool
2026-04-10T17:34:55Z INF ICMP proxy will use 192.168.0.11 as source for IPv4
2026-04-10T17:34:55Z INF Tunnel connection curve preferences: [X25519MLKEM768 CurveP256] connIndex=0 event=0 ip=198.41.200.73
2026-04-10T17:34:55Z INF ICMP proxy will use 2001:e68:449d:e28e:5565:48c:44b6:c516 in zone Wi-Fi 2 as source for IPv6
2026-04-10T17:34:55Z INF Starting metrics server on 127.0.0.1:20241/metrics
2026-04-10T17:34:55Z INF Registered tunnel connection connIndex=0 connection=11bdb0c8-7e58-4624-a613-0dd253c074dd event=0 ip=198.41.200.73 location=sin13 protocol=quic
```

Phase 4: The Integration (Connecting the Dots)

This is the "Networking" part of the project. We tell the Proxy to listen for the Cloudflare URL and forward it to our Docker container.

Configuration

- **Domain Name:** The `.trycloudflare.com` link.
- **Forward Hostname:** `host.docker.internal` (The Docker bridge).
- **Forward Port:** `8080` (The PHP site).



Phase 5: Implementing the WAF (The "Advanced" Shield)

While the Nginx Proxy Manager (NPM) has a default "Block Common Exploits" toggle, it is often too "loose" and can be bypassed by clever encoding or modern JavaScript event handlers. To ensure the **ByteBandit Tech Store** was fully hardened, I implemented custom **Negative Security Logic** using Regular Expressions (Regex) at the application layer.

The Configuration Script

I injected the following logic into the **Advanced** configuration of the Nginx Proxy Host. This allows the WAF to inspect the `$args` (the search query) before the request ever reaches the PHP backend.

```
# 1. Block XSS keywords in the Search Query ($args)
if ($args ~* "(script|alert|onerror|onload|eval|javascrip
```

```

t)") {
    return 403;
}

# 2. Block HTML tags (< >) in the Search Query (Plain text
& Encoded)
if ($args ~* "(<|%3C|%3E|>)" ) {
    return 403;
}

# 3. Block SQL Injection keywords
if ($args ~* "(union|select|insert|update|delete|drop|bench
mark)") {
    return 403;
}

```

Technical Breakdown of the Rules

1. XSS Keyword Interception

This rule targets the **execution** of malicious JavaScript. By using the `~*` operator, the WAF performs a **Case-Insensitive** match. This prevents a "Case-Bypass" where an attacker might try `<sCrIpT>` instead of `<script>`. It specifically looks for:

- **Event Handlers:** `onerror` and `onload` are blocked to stop "Tag-less XSS" where an attacker triggers code via a broken image or a loaded resource.
- **Execution Functions:** `alert` and `eval` are filtered to prevent the execution of arbitrary payloads.

2. HTML & Encoding Shield

A common WAF bypass involves **URL Encoding**. If a filter only looks for the literal `<` character, a hacker can use `%3C` to slip the payload past the firewall.

- This rule protects against both **Literal Tags** (`<`, `>`) and **Hex Encoded Tags** (`%3C`, `%3E`).
- By blocking these characters in the `$args` (query string), we prevent the browser from rendering unauthorized HTML elements on the Tech Store's search results page.

3. SQL Injection (SQLi) Barrier

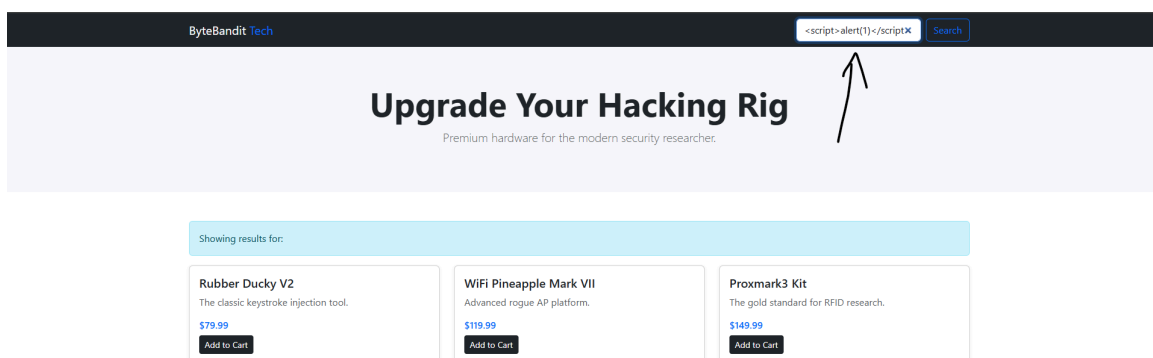
This rule protects the backend database from manipulation. It identifies high-risk SQL keywords that have no business being in a search bar query:

- **union select**: The classic signature of a data-exfiltration attack used to steal user credentials.
- **drop / delete**: Prevents "Destructive SQLi" where an attacker attempts to wipe the product database.
- **benchmark**: Blocks "Time-based Blind SQLi" where attackers use server response delays to map the database structure.

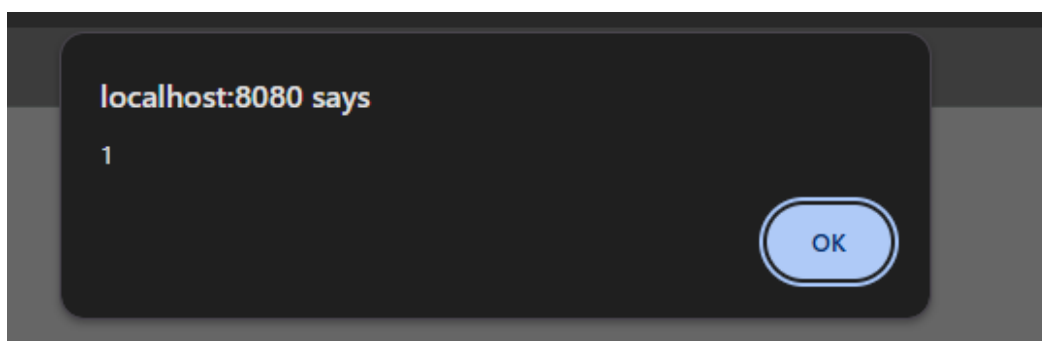
Phase 6: The Proof of Concept (The Comparison)

Scenario A: The Vulnerability (Bypass)

- **Action:** Enter `<script>alert(1)</script>` into `localhost:8080`.



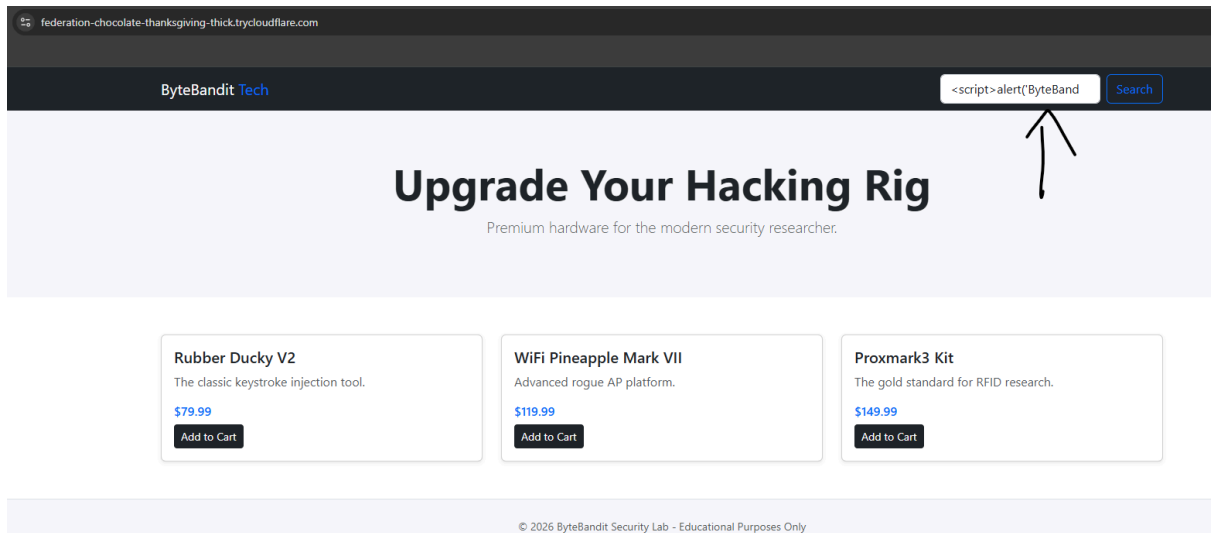
- **Result:** The JavaScript executes (Alert box appears).



- **Meaning:** The application is naturally insecure.

Scenario B: The Protection (WAF Active)

- **Action:** Enter the same script into the **Cloudflare Link**.



- **Result:** A **403 Forbidden** error page.



- **Meaning:** The WAF successfully intercepted the attack.

Conclusion: Project Reflection & Impact

The WAF project successfully demonstrated the transition from a "Default-Deny" mindset to a "Granular-Defense" architecture. By moving security logic from the application code to a dedicated **Layer 7 Reverse Proxy (Nginx)**, I created a more resilient and scalable defense system.

1. Key Achievements

- **Reduced Attack Surface:** Successfully blocked common XSS and SQLi payloads before they reached the backend PHP interpreter.

- **Network Visibility:** Leveraged Cloudflare Tunnels to expose a local development environment securely to the internet for real-world WAF testing.
- **Hardened Logic:** Developed custom Nginx filtering rules that account for **URL encoding** and **case-sensitivity**, closing common WAF bypass loopholes.

Final Thought

This project reinforced a core tenet of cybersecurity: **Defense in Depth**. Even if the application is vulnerable, a properly configured network perimeter can serve as a vital second line of defense.